



OBJECT ORIENTED ANALYSIS AND DESIGN

Ch - 9

Relationships

A study material for the students of GLS University

What is a relationship?

A meaningful connection between UML elements
Shows how different parts of a system are connected

Between actors and use cases → Association

Example: Customer → places order → Order Use Case

Between use cases →

Generalization (one use case is a specialized form of another)

«include» (common behavior reused by other use cases)

«extend» (optional or conditional behavior)

Between actors → Generalization

Example: Admin is a specialized version of User

Between objects → Links

A link connects two objects that can communicate.

Between classes → Associations

Show how objects of one class are connected to objects of another.

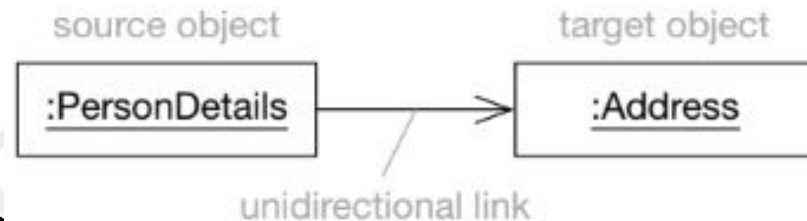
What is a Link?

- A link is a connection between two objects
- Allows objects to send messages to each other

Implementation:

Java → object reference

C++ → pointer or reference



Types of Links:

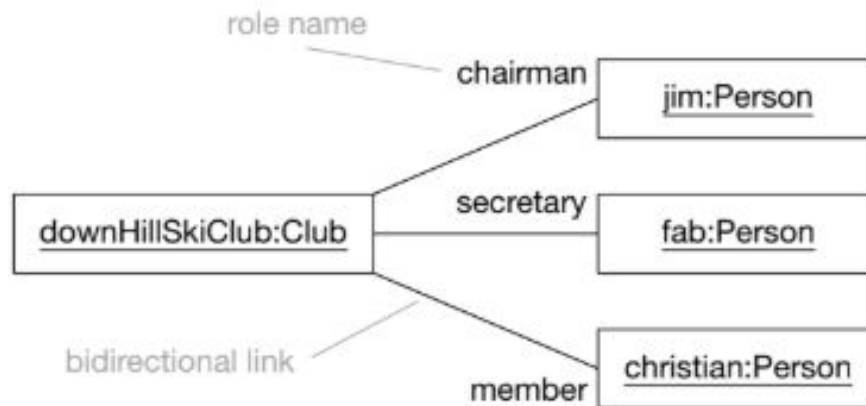
- **Unidirectional** (one-way) - one-way arrow shows which object can access the other.
- **Bidirectional** (two-way) - no arrows; both can access each other.

Object Diagrams

Object diagrams show objects and links at a specific time.

Links are dynamic – they can change during program execution.

Example -



Paths in Diagrams

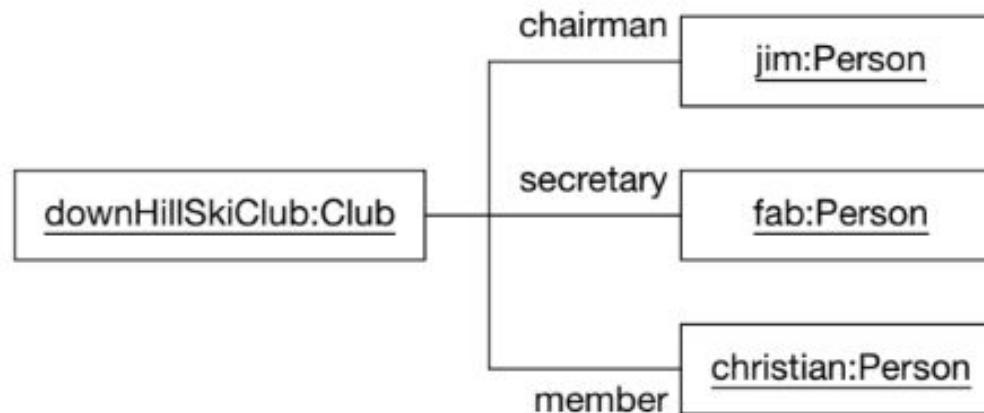
Paths = lines connecting elements in UML diagrams

Styles:

Orthogonal → straight horizontal/vertical lines

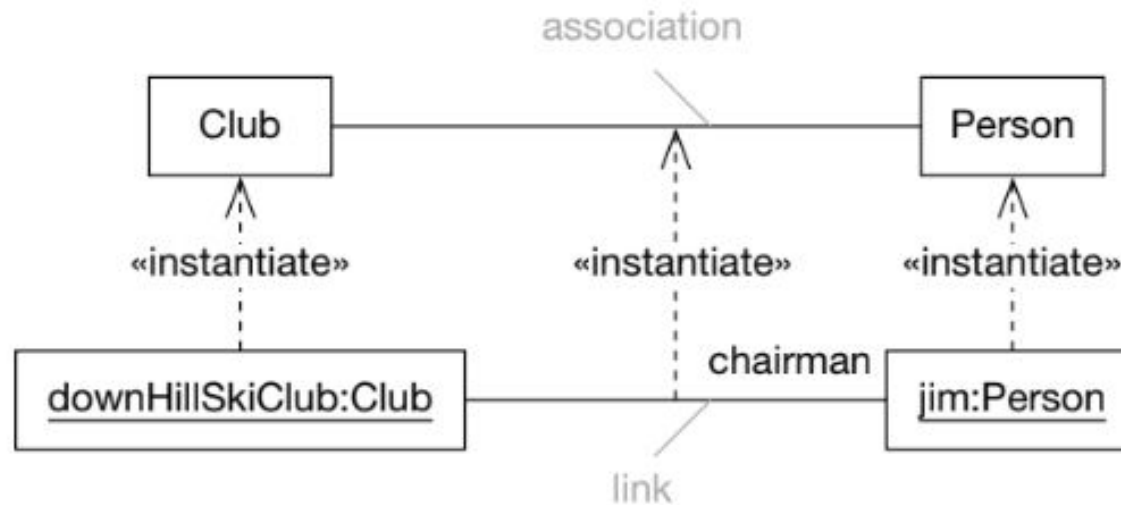
Oblique → slanted lines

Rule: diagrams should always be neat and clear



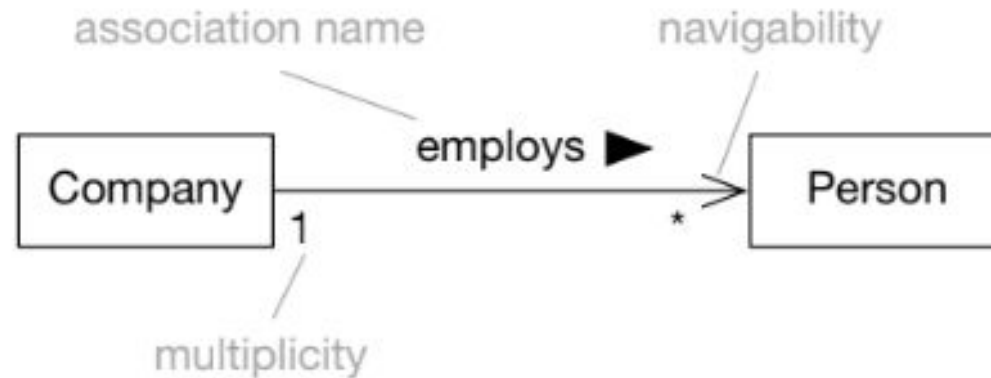
What is an association?

- Associations are relationships between classes
- Links connect objects; associations connect classes
- A link = instantiation of an association
- Objects ↔ Classes | Links ↔ Associations



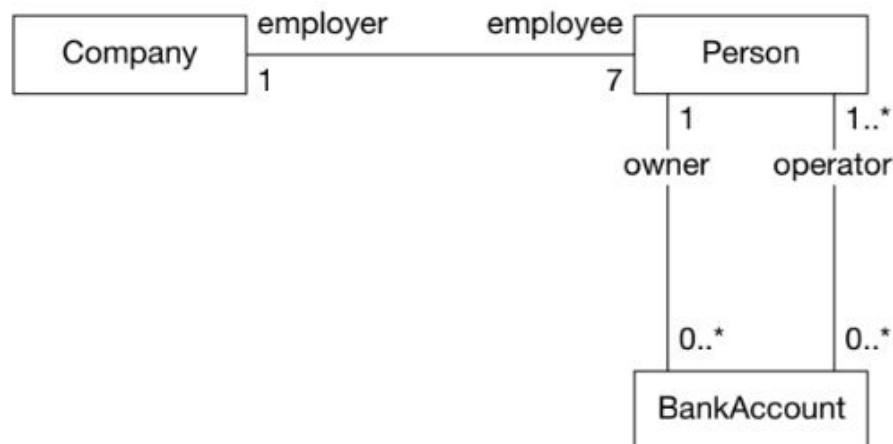
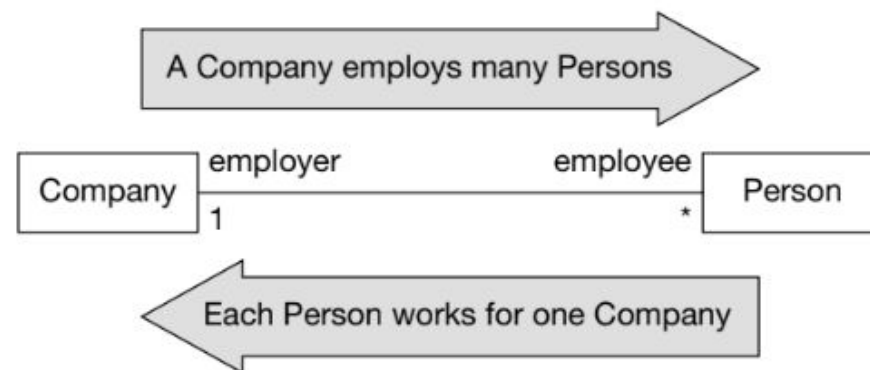
Association Syntax

- Can include association name, role names, multiplicity, navigability
- Example: A Company employs many Persons



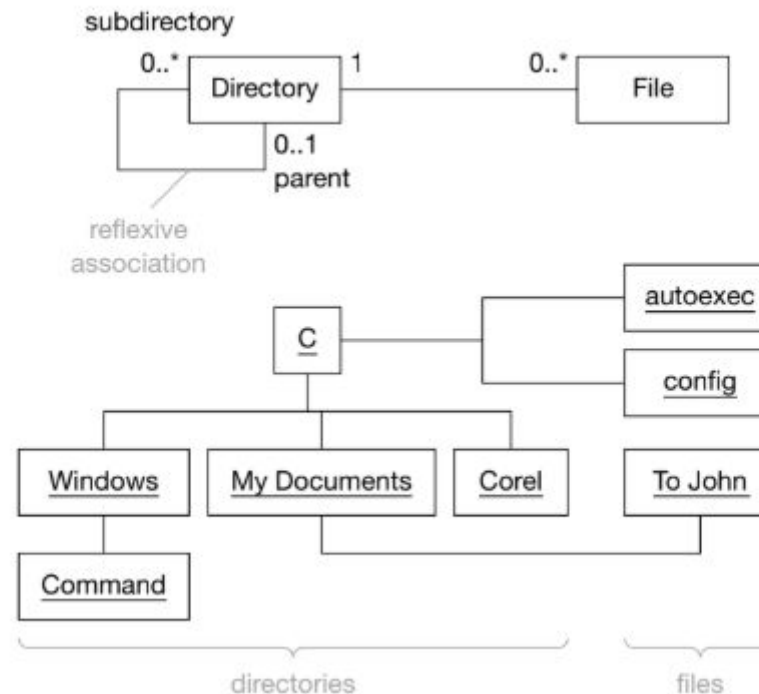
Multiplicity

- Defines how many objects can participate in a relationship
- $0..1 \rightarrow$ Zero or one
- $1 \rightarrow$ Exactly one
- $0..*$ or $*$ $\rightarrow$ Zero or many
- $1..*$ $\rightarrow$ One or many



Reflexive Associations

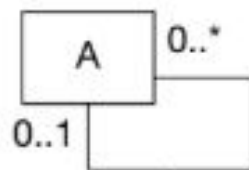
- Class linked to itself
- Example: Directory has subdirectories (0..*)
- Each Directory may have one parent (0..1)



Hierarchies vs Networks

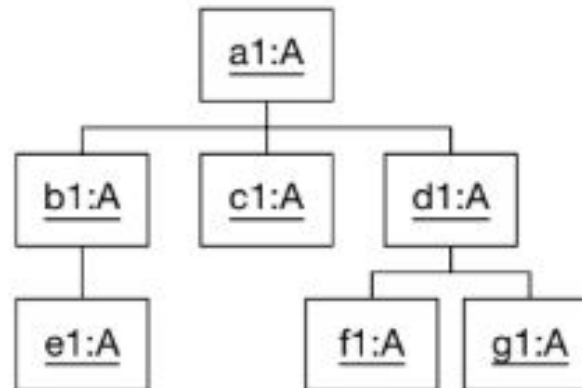
- Hierarchy: one root, rigid structure (e.g., directory tree)
- Network: no fixed root, flexible connections (e.g., WWW)

Class diagram



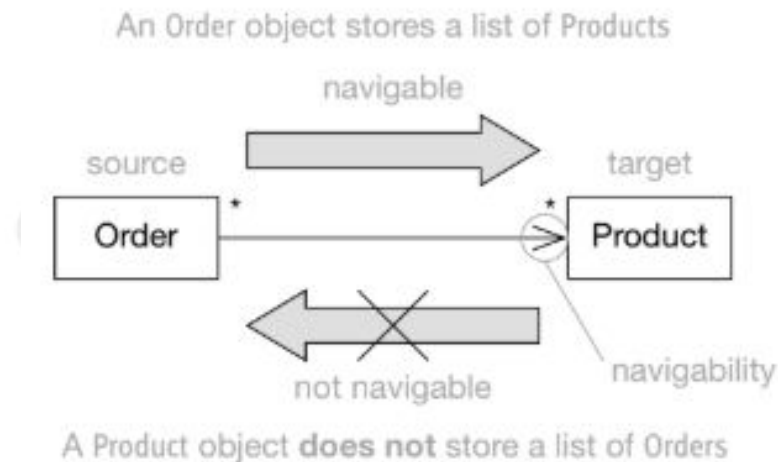
association
hierarchy

Example object diagram



Navigability

- Defines direction of message passing
- Example: Order → Product (Order knows Products)
- Product ✕ Order (Product doesn't know Orders)
- Helps minimize coupling



Associations & Attributes

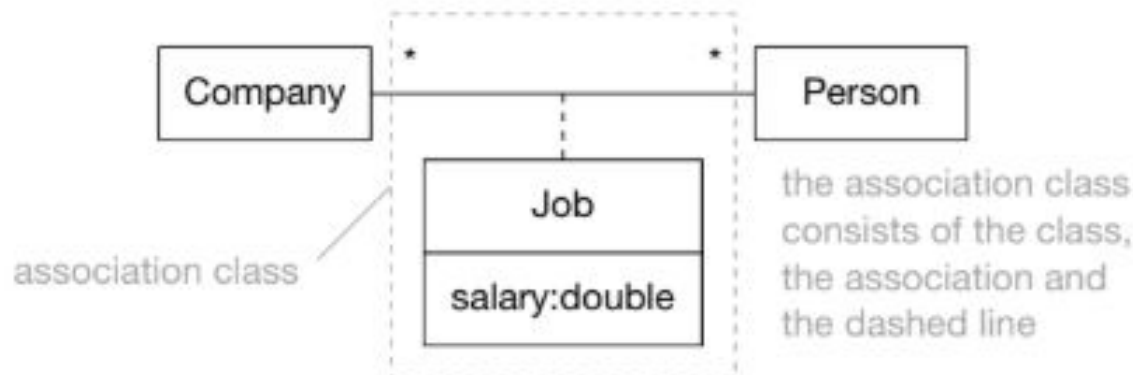
- Association $\approx$ pseudo-attribute
- Example: House has address:Address
- Multiplicity $> 1 \rightarrow$ usually model as association
- Multiplicity $= 1 \rightarrow$ may be modeled as attribute



If a navigable relationship has a role name, then it is as though the source class has a pseudo-attribute with the same name as the role name and the same type as the target class

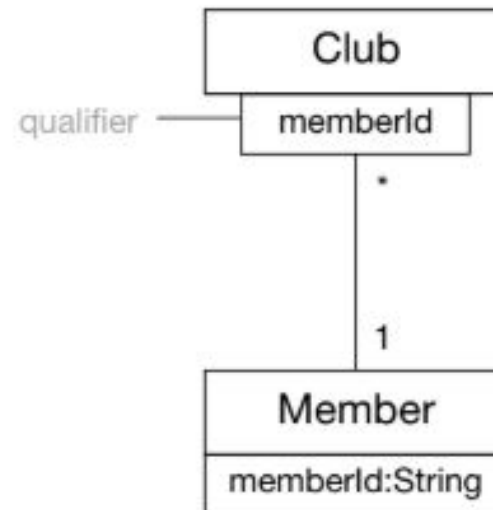
Association Classes

- Used in many-to-many with extra attributes
- Example: Person ↔ Company with Job (salary)
- If multiple jobs possible → reify as normal class



Qualified Associations

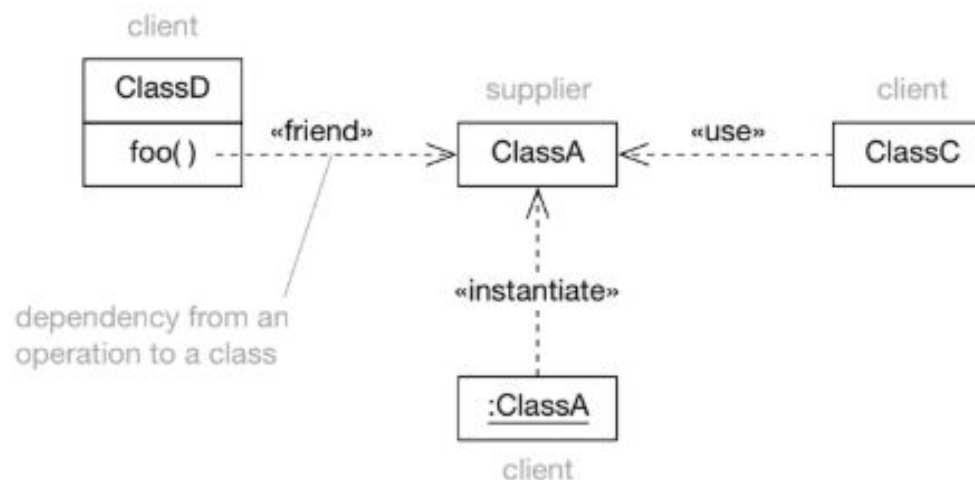
- Narrow a many-to-many into many-to-one using unique key
- Example: Club → Member (qualified by memberId)



the combination {Club, memberId}
specifies a unique target

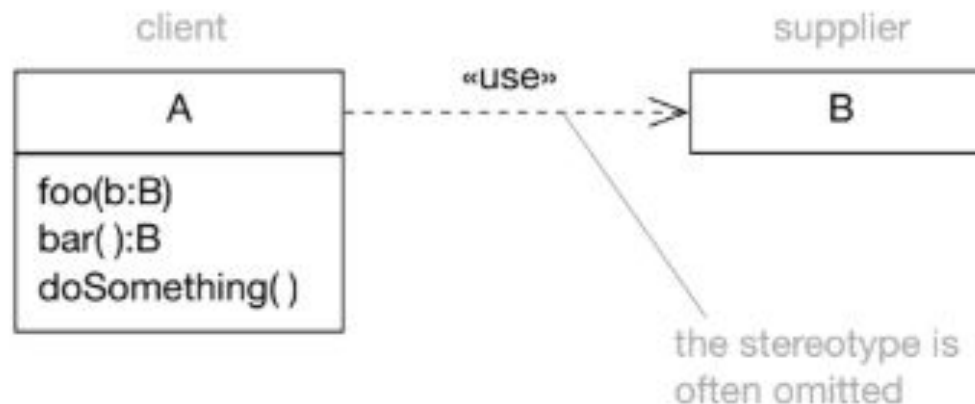
What is a Dependency?

- Relationship where client depends on supplier
- Change in supplier may affect client
- Not always association (e.g., passing objects as parameters)



Common Usage Dependency

- «use»: client class uses supplier class
- Example: A method in Class A creates or returns an object of Class B



THANK YOU